Introdução ao Git

Branches e Fluxo de Trabalho com Git

O que são Branches no Git?

- **Branches** (ou ramificações) permitem trabalhar em diferentes versões do código sem afetar a versão principal.
- Útil para **desenvolvimento paralelo**, **correções de bugs** e **novas funcionalidades**.
- O branch principal normalmente é chamado de **main** (ou **master**).

```
git branch                    # Lista os branches

git branch nova-feature       # Cria um novo branch

git checkout nova-feature     # Troca para o branch criado
```

📌 **Dica:** A partir do Git 2.23, use:

```
git switch -c nova-feature    # Alternativa para criar e trocar de branch
```

Criando e Trocando de Branches

- Criar um branch e alternar para ele:

```
git checkout -b minha-feature
```

- Listar todos os branches do repositório:

```
git branch
```

- Alternar entre branches existentes:

```
git checkout main
```

- Enviar um branch remoto para o GitHub:

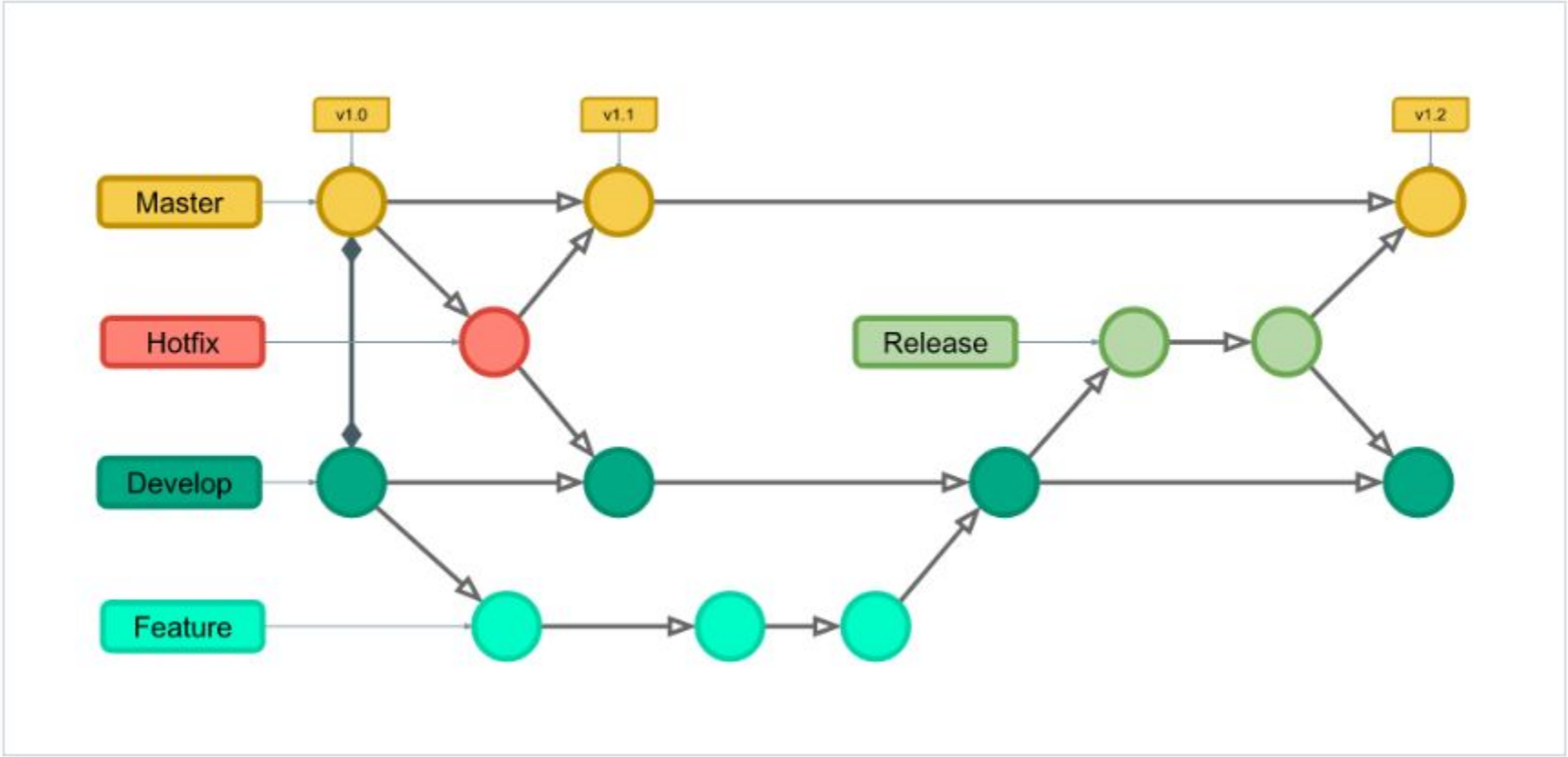
```
git push origin minha-feature
```

📌 **Dica:** No GitHub, branches são usados para organizar diferentes estágios do desenvolvimento.

Introdução ao GitFlow

- **GitFlow** é um fluxo de trabalho baseado em branches para organizar o desenvolvimento.
- Principais branches:
  - **main** → Código estável e pronto para produção.
  - **develop** → Contém o código em desenvolvimento.
  - **feature branches** → Para desenvolvimento de novas funcionalidades.
  - **release branches** → Para preparar uma nova versão.
  - **hotfix branches** → Para corrigir problemas urgentes em produção.

📌 **Vantagem:** Mantém um fluxo organizado para múltiplos desenvolvedores.



ve
em
co
có
po

Fluxo de Trabalho com GitFlow

- Criar uma nova feature

```
git checkout -b feature/minha-nova-feature develop
```

- Finalizar e integrar a feature

```
git checkout develop
```

```
git merge feature/minha-nova-feature
```

- Criar um branch de release

```
git checkout -b release/1.0 develop
```

- Corrigir bugs críticos na produção (hotfix)

```
git checkout -b hotfix/1.0.1 main
```

📌 Dica: No GitHub, Pull Requests ajudam a revisar e mesclar código de branches.

Boas Práticas na Gestão de Branches

- **Mantenha a branch `main` estável e sempre funcionando.**
- **Crie branches curtas** para evitar conflitos ao mesclar (merge).
- **Use nomes descritivos** para branches:
  a. `feature/autenticacao-google`
  b. `hotfix/corrige-bug-login`
- **Sempre atualize sua branch antes de enviar código:**

  ```
  git pull origin main
  ```

- **Revise código antes de fazer merge (Pull Request no GitHub).**
- **Apague branches desnecessárias** após o merge:

  ```
  git branch -d minha-feature
  ```

Resumo e Próximos Passos

✅ Branches ajudam a organizar o desenvolvimento.

✅ GitFlow melhora a colaboração em equipe.

✅ Boas práticas evitam conflitos e facilitam a gestão do código.

📌 Prática: Vamos criar uma nova branch em nosso repositório compartilhado no Github. Para o nome da branch, use `nome-sobrenome`. Nessa branch, adicione no arquivo chamado README.md o seu nome e envie a branch para o repositório remoto.

https://github.com/hamc/fatec20261

Dúvidas?